

Лабораторна робота 7

Автентифікація та авторизація

Метою лабораторної роботи є набуття практичних навичок реалізації автентифікації та авторизації у Node.js-застосунках шляхом розширення застосунку з лабораторної роботи №6, зокрема:

- хешування паролів з використанням `bcryptjs` та автоматичної генерації солі;
- видача та верифікація JWT-токенів за схемою `access + refresh`;
- збереження токенів у `httpOnly cookies` та керування їх життєвим циклом;
- захист CRUD-маршрутів `middleware`-ом автентифікації та перевірка власника запису.

Хід роботи

Завдання 1. Реєстрація користувачів з хешуванням пароля

Для безпечного хешування було встановлено бібліотеку `bcryptjs`.

Далі було створено Mongoose-модель `User`, яка буде представляти сутність користувача.

Лістинг `src/models/User.ts`:

```
import mongoose from "mongoose";
import bcrypt from "bcryptjs";

const userSchema = new mongoose.Schema({
  email: {
    type: String,
    required: true,
    unique: true,
  },
});
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.7						
Змн.	Арк.	№ докум.	Підпис	Дата	Звіт з лабораторної роботи №7			Літ.	Арк.	Аркушів	
Розроб.		Сергійко В.М.									
Перевір.		Оринчак А. І.								1	22
Реценз.								ФІКТ, гр. ІПЗк-24			
Н. Контр.											
Зав.каф.											

```

password: {
  type: String,
  required: true,
},
createdAt: {
  type: Date,
  default: Date.now,
},
});

userSchema.pre("save", async function () {
  if (!this.isModified("password")) return;

  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
});

export const User = mongoose.model("User", userSchema);

```

Далі додаємо ендпоінт реєстрації.

Лістинг src/routes/auth.ts:

```

import express from "express";
import { z } from "zod";
import { User } from "../models/User";

const router = express.Router();

const registerSchema = z.object({
  email: z.string().email("Некоректний формат пошти"),
  password: z.string().min(6, "Мінімальна довжина пароля - 6 символів"),
});

router.post("/register", async (req, res) => {
  try {
    const { email, password } = registerSchema.parse(req.body);

    const existingUser = await User.findOne({ email });
    if (existingUser) {
      return res
        .status(409)
        .json({ message: "Користувач з такою поштою вже існує" });
    }

    const user = new User({ email, password });
    await user.save();

    res.status(201).json({
      id: user._id,
      email: user.email,

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.7	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		2

```

        createdAt: user.createdAt,
    });
} catch (error) {
    if (error instanceof z.ZodError) {
        return res.status(400).json({ errors: error.issues });
    }
    console.error(error);
    res.status(500).json({ message: "Внутрішня помилка сервера" });
}
});

export default router;

```

Покриємо ендпоінт інтеграційними тестами та перевіримо виконання тестів.

Лістинг tests/auth.test.ts:

```

import request from "supertest";
import { MongoMemoryServer } from "mongodb-memory-server";
import mongoose from "mongoose";
import app from "../src/app";
import {
    describe,
    it,
    expect,
    beforeAll,
    afterAll,
    afterEach,
    jest,
    beforeEach,
} from "@jest/globals";
import { User } from "../src/models/User";

let mongoServer: MongoMemoryServer;

beforeAll(async () => {
    mongoServer = await MongoMemoryServer.create();
    const mongoUri = mongoServer.getUri();
    await mongoose.connect(mongoUri);
});

afterAll(async () => {
    await mongoose.disconnect();
    await mongoServer.stop();
});

beforeEach(async () => {
    await User.deleteMany({});
});

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.7	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		3

```

describe("POST /auth/register", () => {
  it("повинен успішно реєструвати нового користувача", async () => {
    const res = await request(app).post("/auth/register").send({
      email: "test@example.com",
      password: "password123",
    });

    expect(res.status).toBe(201);
    expect(res.body).toHaveProperty("id");
    expect(res.body.email).toBe("test@example.com");

    expect(res.body).not.toHaveProperty("password");

    const userInDb = await User.findOne({ email: "test@example.com" });
    expect(userInDb).not.toBeNull();
    expect(userInDb?.password).not.toBe("password123");
  });

  it("повинен повертати 409 Conflict при дублюванні пошти", async () => {
    await User.create({
      email: "test@example.com",
      password: "hashedpassword",
    });

    const res = await request(app).post("/auth/register").send({
      email: "test@example.com",
      password: "newpassword123",
    });

    expect(res.status).toBe(409);
  });

  it("повинен повертати 400 при невалідному email", async () => {
    const res = await request(app).post("/auth/register").send({
      email: "invalid-email",
      password: "password123",
    });

    expect(res.status).toBe(400);
  });

  it("повинен повертати 400 при надто короткому паролі", async () => {
    const res = await request(app).post("/auth/register").send({
      email: "test@example.com",
      password: "123",
    });

    expect(res.status).toBe(400);
  });
});

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.7	Арк.
						4
Змн.	Арк.	№ докум.	Підпис	Дата		

```
PS D:\Study\Politech\25-26\2_semestr\NodeJs\Lab5\dotaHeroes> npm run test

> lab3@1.0.0 test
> jest

PASS tests/hero.model.test.ts (7.989 s)
PASS tests/auth.test.ts (8.349 s)
PASS tests/hero.test.ts (8.441 s)

Test Suites: 3 passed, 3 total
Tests:       27 passed, 27 total
Snapshots:   0 total
Time:        9.062 s
Ran all test suites.
```

Рис.7.1 – Результат тестування

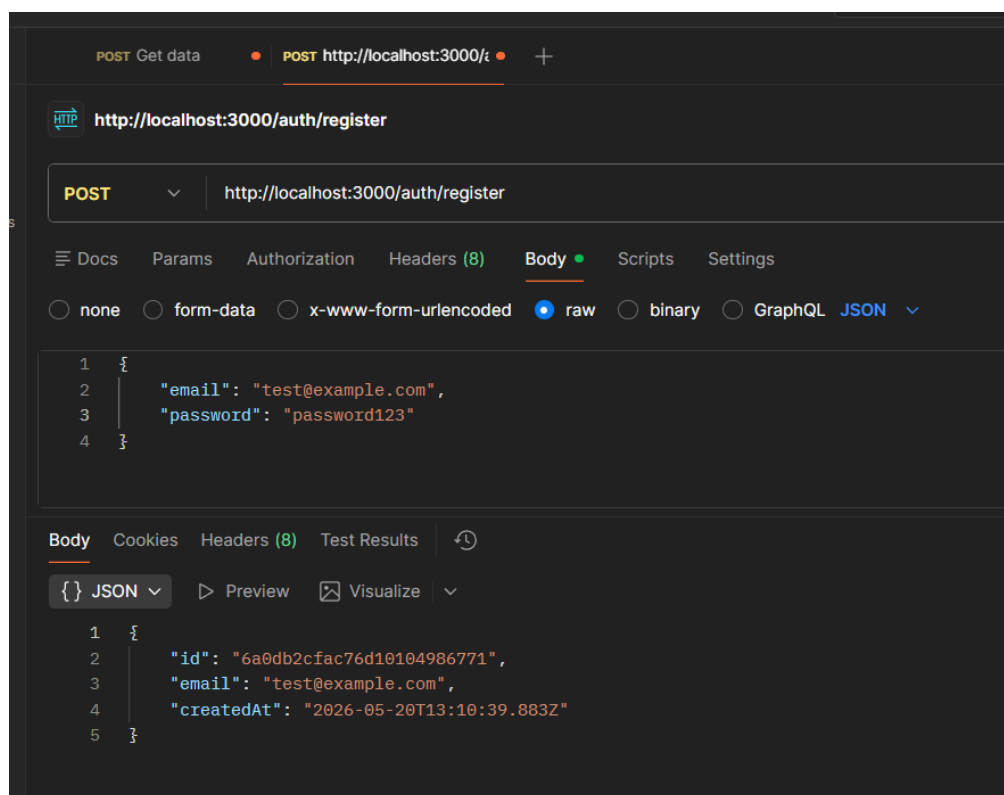


Рис.7.2 – Реєстрація користувача

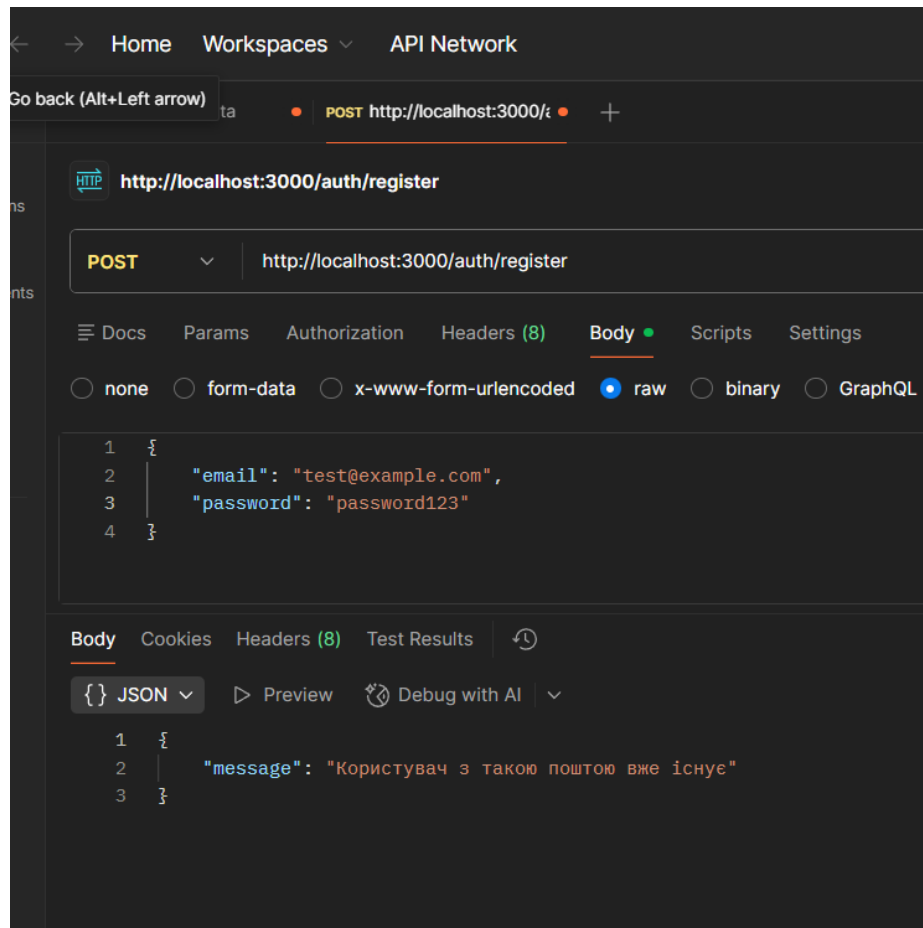


Рис.7.3 – Реєстрація користувача з уже наявною поштою

Завдання 2. Логін з access та refresh токенами

Встановимо пакети jsonwebtoken та @types/jsonwebtoken. Додамо змінну JWT_SECRET у .env для локальної розробки та задамо її через fly secrets set для сервера.

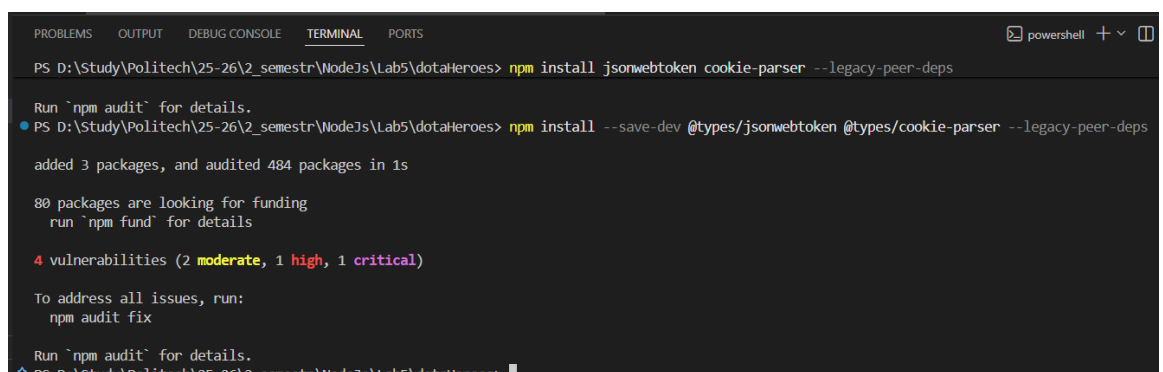


Рис.7.4 – Встановлення пакетів

Реалізуємо ендпойнти /auth/login, /auth/refresh, та /auth/logout.

Лістинг src/routes/auth.ts:

```
import express from "express";
import { z } from "zod";
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.7	Арк.
						6
Змн.	Арк.	№ докум.	Підпис	Дата		

```

import { User } from "../models/User";
import jwt from "jsonwebtoken";
import bcrypt from "bcryptjs";

const router = express.Router();

const registerSchema = z.object({
  email: z.string().email("Некоректний формат пошти"),
  password: z.string().min(6, "Мінімальна довжина пароля - 6 символів"),
});

router.post("/register", async (req, res) => {
  try {
    const { email, password } = registerSchema.parse(req.body);

    const existingUser = await User.findOne({ email });
    if (existingUser) {
      return res
        .status(409)
        .json({ message: "Користувач з такою поштою вже існує" });
    }

    const user = new User({ email, password });
    await user.save();

    res.status(201).json({
      id: user._id,
      email: user.email,
      createdAt: user.createdAt,
    });
  } catch (error) {
    if (error instanceof z.ZodError) {
      return res.status(400).json({ errors: error.issues });
    }
    console.error(error);
    res.status(500).json({ message: "Внутрішня помилка сервера" });
  }
});

const loginSchema = z.object({
  email: z.string().email(),
  password: z.string(),
});

const generateTokens = (userId: string) => {
  const accessToken = jwt.sign({ userId }, process.env.JWT_SECRET as string, {
    expiresIn: "15m",
  });
  const refreshToken = jwt.sign({ userId }, process.env.JWT_SECRET as string, {
    expiresIn: "30d",
  });

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.7	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		7

```

});
return { accessToken, refreshToken };
};

const cookieOptions = {
  httpOnly: true,
  secure: process.env.NODE_ENV === "production",
  sameSite: "strict" as const,
};

router.post("/login", async (req, res) => {
  try {
    const { email, password } = loginSchema.parse(req.body);

    const user = await User.findOne({ email });
    if (!user) {
      return res.status(401).json({ message: "Невірний email або пароль" });
    }

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
      return res.status(401).json({ message: "Невірний email або пароль" });
    }

    const { accessToken, refreshToken } = generateTokens(user.id);

    res.cookie("access_token", accessToken, cookieOptions);
    res.cookie("refresh_token", refreshToken, cookieOptions);

    res.status(200).json({ message: "Успішний вхід" });
  } catch (error) {
    res.status(400).json({ message: "Помилка валідації вхідних даних" });
  }
});

router.post("/refresh", async (req, res) => {
  const { refresh_token } = req.cookies;

  if (!refresh_token) {
    return res.status(401).json({ message: "Токен відсутній" });
  }

  try {
    const decoded = jwt.verify(
      refresh_token,
      process.env.JWT_SECRET as string,
    ) as { userId: string };

    const { accessToken, refreshToken } = generateTokens(decoded.userId);

    res.cookie("access_token", accessToken, cookieOptions);
  }
});

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.7	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		8

```

    res.cookie("refresh_token", refreshToken, cookieOptions);

    res.status(200).json({ message: "Токени успішно оновлено" });
  } catch (error) {
    res.status(401).json({ message: "Недійсний або прострочений токен" });
  }
});

router.post("/logout", (req, res) => {
  res.clearCookie("access_token");
  res.clearCookie("refresh_token");
  res.status(200).json({ message: "Вихід успішний" });
});

export default router;

```

Оновимо тести для наших нових ендпойнтів.

```

describe("POST /auth/login", () => {
  it("повинен успішно логінити та повертати cookies з токенами", async () => {
    const res = await request(app).post("/auth/login").send({
      email: testUser.email,
      password: testUser.password,
    });

    expect(res.status).toBe(200);
    expect(res.body.message).toBe("Успішний вхід");

    cookies = res.headers["set-cookie"] as unknown as string[];
    expect(cookies).toBeDefined();

    const hasAccessToken = cookies.some((cookie) =>
      cookie.includes("access_token="),
    );
    const hasRefreshToken = cookies.some((cookie) =>
      cookie.includes("refresh_token="),
    );
    expect(hasAccessToken).toBeTruthy();
    expect(hasRefreshToken).toBeTruthy();
  });

  it("повинен повертати 401 при неправильному паролі", async () => {
    const res = await request(app).post("/auth/login").send({
      email: testUser.email,
      password: "wrongpassword",
    });

    expect(res.status).toBe(401);
    expect(res.body.message).toBe("Невірний email або пароль");
    expect(res.headers["set-cookie"]).toBeUndefined();
  });
});

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.7	Арк.
						9
Змн.	Арк.	№ докум.	Підпис	Дата		

```

it("повинен повертати 401 при неіснуючому email", async () => {
    const res = await request(app).post("/auth/login").send({
        email: "nonexistent@example.com",
        password: testUser.password,
    });

    expect(res.status).toBe(401);
});

describe("POST /auth/refresh", () => {
    it("повинен оновлювати токени при наявності валідного refresh_token", async () => {
        const loginRes = await request(app).post("/auth/login").send(testUser);

        const validCookies = loginRes.headers[
            "set-cookie"
        ] as unknown as string[];

        const refreshRes = await request(app)
            .post("/auth/refresh")
            .set("Cookie", validCookies);

        expect(refreshRes.status).toBe(200);
        expect(refreshRes.headers["set-cookie"]).toBeDefined();

        const newCookies = refreshRes.headers[
            "set-cookie"
        ] as unknown as string[];
        const hasNewAccessToken = newCookies.some((c: string) =>
            c.includes("access_token="),
        );
        expect(hasNewAccessToken).toBeTruthy();
    });

    it("повинен повертати 401, якщо refresh_token відсутній", async () => {
        const res = await request(app).post("/auth/refresh");

        expect(res.status).toBe(401);
        expect(res.body.message).toBe("Токен відсутній");
    });

    it("повинен повертати 401, якщо refresh_token недійсний", async () => {
        const res = await request(app)
            .post("/auth/refresh")
            .set("Cookie", ["refresh_token=invalid_token_string; HttpOnly"]);

        expect(res.status).toBe(401);
        expect(res.body.message).toBe("Недійсний або прострочений токен");
    });
});

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.7	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		10

```
});

describe("POST /auth/logout", () => {
  it("повинен очищати cookies при виході", async () => {
    const res = await request(app).post("/auth/logout");

    expect(res.status).toBe(200);
    expect(res.body.message).toBe("Вихід успішний");

    const clearCookies = res.headers["set-cookie"];
    expect(clearCookies).toBeDefined();
    expect(clearCookies[0]).toMatch(/access_token=/);
    expect(clearCookies[1]).toMatch(/refresh_token=/);
  });
});
```

```
PS D:\Study\Politech\25-26\2_semestr\NodeJs\Lab5\dotaHeroes> npm run test

PASS tests/hero.test.ts (5.692 s)
PASS tests/auth.test.ts (6.397 s)

Test Suites: 3 passed, 3 total
Tests:       34 passed, 34 total
Snapshots:   0 total
Time:        6.895 s
Ran all test suites.
```

Рис.7.5 – Проходження тестів

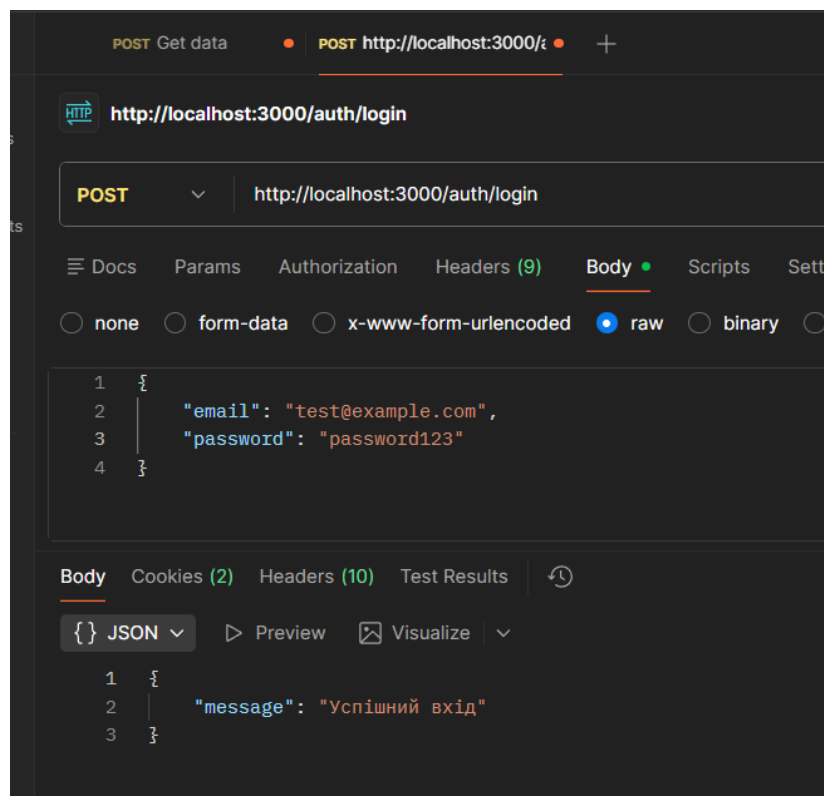


Рис.7.6 – Перевірка авторизації

Завдання 3. Захищені маршрути та авторизація власника

Реалізуємо middleware `requireAuth`: зчитує `access_token` з cookie, верифікує підпис та строк дії, у разі успіху прикріплює інформацію про користувача (принаймні `userId`) до `req` (через розширення типу `Request`). У разі невдачі повертає 401 `Unauthorized` без подробиць у тілі відповіді.

Лістинг `src/middleware/requireAuth.ts`:

```
import { Request, Response, NextFunction } from "express";
import jwt from "jsonwebtoken";

export interface AuthRequest extends Request {
  userId?: string;
}

export const requireAuth = (
  req: AuthRequest,
  res: Response,
  next: NextFunction,
): void => {
  const token = req.cookies.access_token;

  if (!token) {
    res.status(401).json({ message: "Unauthorized" });
    return;
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET as string) as {
      userId: string;
    };

    req.userId = decoded.userId;
    next();
  } catch (error) {
    res.status(401).json({ message: "Unauthorized" });
    return;
  }
};
```

Застосуємо `requireAuth` до маршрутів, що змінюють стан: `POST`, `PUT`, `DELETE` для ресурсу з лаб. 3–4. У `PUT` та `DELETE` перед застосуванням змін перевіряємо, що `ownerId` запису збігається з `req.userId`. У разі невідповідності повертатимемо 403 `Forbidden`. Та додамо у `Mongoose`-схему ресурсу поле `ownerId`.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.7	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		12

```

52     },
53     roles: {
54       type: [String],
55       required: true,
56       validate: {
57         validator: function (v: string[]) {
58           return v && v.length > 0;
59         },
60         message: "Герой повинен мати хоча б одну роль",
61       },
62     },
63     ownerId: {
64       type: mongoose.Schema.Types.ObjectId,
65       ref: "User",
66       required: true,
67     },
68   },
69   {
70     timestamps: true,
71     toJSON: { virtuals: true },
72     toObject: { virtuals: true },
73   },
74 );

```

Рис.7.7 – Додане поле ownerId

Лістинг routes/hero.ts:

```

import { Router, Request, Response, NextFunction } from "express";
import { heroStorage } from "../storage/hero";
import { createHeroSchema, updateHeroSchema } from "../schemas/hero.schema";
import { validate } from "../middleware/validate";
import { requireAuth, AuthRequest } from "../middleware/requireAuth";

const router = Router();

router.get(
  "/melee",
  async (req: Request, res: Response, next: NextFunction) => {
    try {
      const result = await heroStorage.getAll({
        attackType: "Melee",
        limit: 100,
      });
      res.status(200).json(result.data);
    } catch (error) {
      next(error);
    }
  },
);

router.get("/", async (req: Request, res: Response, next: NextFunction) => {
  try {

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.7	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		13

```

    const result = await heroStorage.getAll(req.query as any);
    res.status(200).json(result);
  } catch (error) {
    next(error);
  }
});

router.get(
  "/:id",
  async (req: Request<{ id: string }>, res: Response, next: NextFunction) => {
    try {
      const hero = await heroStorage.getById(req.params.id);
      if (!hero) {
        res.status(404).json({ message: "Hero not found" });
        return;
      }
      res.status(200).json(hero);
    } catch (error) {
      next(error);
    }
  },
);

router.post(
  "/",
  requireAuth,
  validate(createHeroSchema),
  async (req: Request, res: Response, next: NextFunction) => {
    try {
      const authReq = req as AuthRequest;

      const heroData = {
        ...req.body,
        ownerId: authReq.userId,
      };

      const newHero = await heroStorage.create(heroData);
      res.status(201).json(newHero);
    } catch (error) {
      next(error);
    }
  },
);

router.patch(
  "/:id",
  requireAuth,
  validate(updateHeroSchema),
  async (req: Request<{ id: string }>, res: Response, next: NextFunction) => {
    try {
      const authReq = req as AuthRequest;

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.7	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		14

```

    const hero = await heroStorage.getById(req.params.id);
    if (!hero) {
        res.status(404).json({ message: "Hero not found" });
        return;
    }

    if (hero.ownerId?.toString() !== authReq.userId) {
        res.status(403).json({ message: "Forbidden" });
        return;
    }

    const updatedHero = await heroStorage.update(req.params.id, req.body);
    res.status(200).json(updatedHero);
} catch (error) {
    next(error);
}
},
);

router.delete(
    "/:id",
    requireAuth,
    async (req: Request<{ id: string }>, res: Response, next: NextFunction) => {
        try {
            const authReq = req as AuthRequest;

            const hero = await heroStorage.getById(req.params.id);
            if (!hero) {
                res.status(404).json({ message: "Hero not found" });
                return;
            }

            if (hero.ownerId?.toString() !== authReq.userId) {
                res.status(403).json({ message: "Forbidden" });
                return;
            }

            await heroStorage.remove(req.params.id);
            res.status(204).send();
        } catch (error) {
            next(error);
        }
    },
);

export default router;

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.7	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		15

Покриємо інтеграційними тестами оновлений код

Лістинг hero.auth.test.ts:

```
import request from "supertest";
import { MongoMemoryServer } from "mongodb-memory-server";
import mongoose from "mongoose";
import jwt from "jsonwebtoken";
import app from "../src/app";
import { User } from "../src/models/User";
import { HeroModel } from "../src/models/hero.model";
import {
  describe,
  it,
  expect,
  beforeAll,
  afterAll,
  afterEach,
  jest,
  beforeEach,
} from "@jest/globals";

process.env.JWT_SECRET = "super_secret_test_key_123";

let mongoServer: MongoMemoryServer;

beforeAll(async () => {
  mongoServer = await MongoMemoryServer.create();
  const mongoUri = mongoServer.getUri();
  await mongoose.connect(mongoUri);
});

afterAll(async () => {
  await mongoose.disconnect();
  await mongoServer.stop();
});

describe("Protected Hero Routes (CRUD & Authorization)", () => {
  let user1Id: string;
  let user2Id: string;
  let user1Cookie: string;
  let user2Cookie: string;
  let heroId: string;

  const validHeroData = {
    name: "Anti-Mage",
    attackType: "Melee",
    primaryAttribute: "Agility",
    roles: ["Carry", "Escape"],
  };
});
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.7	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		16

```

beforeEach(async () => {
  await User.deleteMany({});
  await HeroModel.deleteMany({});

  const user1 = await User.create({
    email: "user1@example.com",
    password: "password123",
  });
  const user2 = await User.create({
    email: "user2@example.com",
    password: "password123",
  });

  user1Id = user1._id.toString();
  user2Id = user2._id.toString();

  const token1 = jwt.sign(
    { userId: user1Id },
    process.env.JWT_SECRET as string,
    { expiresIn: "15m" },
  );
  const token2 = jwt.sign(
    { userId: user2Id },
    process.env.JWT_SECRET as string,
    { expiresIn: "15m" },
  );

  user1Cookie = `access_token=${token1}`;
  user2Cookie = `access_token=${token2}`;
});

describe("POST /", () => {
  it("повинен повертати 401, якщо користувач не авторизований (немає токена)",
  async () => {
    const res = await request(app).post("/").send(validHeroData);

    expect(res.status).toBe(401);
  });

  it("повинен створювати героя і прив'язувати його до ownerId", async () => {
    const res = await request(app)
      .post("/")
      .set("Cookie", [user1Cookie])
      .send(validHeroData);

    expect(res.status).toBe(201);
    expect(res.body).toHaveProperty("ownerId");
    expect(res.body.ownerId).toBe(user1Id);

    const heroInDb = await HeroModel.findById(res.body._id);
    expect(heroInDb?.ownerId?.toString()).toBe(user1Id);
  });
});

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.7	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		17

```

    });
  });

describe("PATCH /:id", () => {
  beforeEach(async () => {
    const newHero = await HeroModel.create({
      ...validHeroData,
      ownerId: user1Id,
    });
    heroId = newHero._id.toString();
  });

  it("повинен повертати 401 для неавторизованих запитів", async () => {
    const res = await request(app)
      .patch(`/${heroId}`)
      .send({ name: "Magina" });

    expect(res.status).toBe(401);
  });

  it("повинен оновлювати героя, якщо запит робить власник (ownerId збігається)",
  async () => {
    const res = await request(app)
      .patch(`/${heroId}`)
      .set("Cookie", [user1Cookie])
      .send({ name: "Magina" });

    expect(res.status).toBe(200);
    expect(res.body.name).toBe("Magina");
  });

  it("повинен повертати 403 Forbidden, якщо запит робить НЕ власник", async () => {
    const res = await request(app)
      .patch(`/${heroId}`)
      .set("Cookie", [user2Cookie])
      .send({ name: "Hacked Name" });

    expect(res.status).toBe(403);
    expect(res.body.message).toBe("Forbidden");

    const heroInDb = await HeroModel.findById(heroId);
    expect(heroInDb?.name).toBe("Anti-Mage");
  });
});

describe("DELETE /:id", () => {
  beforeEach(async () => {
    const newHero = await HeroModel.create({
      ...validHeroData,
      ownerId: user1Id,
    });
  });
});

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.7	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		18

```

    heroId = newHero._id.toString();
  });

  it("повинен повертати 401 для неавторизованих запитів", async () => {
    const res = await request(app).delete(`/${heroId}`);
    expect(res.status).toBe(401);
  });

  it("повинен видаляти героя, якщо запит робить власник", async () => {
    const res = await request(app)
      .delete(`/${heroId}`)
      .set("Cookie", [user1Cookie]);

    expect(res.status).toBe(204);

    const heroInDb = await HeroModel.findById(heroId);
    expect(heroInDb).toBeNull();
  });

  it("повинен повертати 403 Forbidden, якщо видалити намагається НЕ власник", async
() => {
    const res = await request(app)
      .delete(`/${heroId}`)
      .set("Cookie", [user2Cookie]);

    expect(res.status).toBe(403);

    const heroInDb = await HeroModel.findById(heroId);
    expect(heroInDb).not.toBeNull();
  });
});
});
});

```

```

PS D:\Study\Politech\25-26\2_semestr\NodeJs\Lab5\dotaHeroes> npm run test

PASS tests/hero.model.test.ts
PASS tests/auth.test.ts
PASS tests/hero.auth.test.ts (6.293 s)

Test Suites: 1 skipped, 3 passed, 3 of 4 total
Tests:       19 skipped, 23 passed, 42 total
Snapshots:   0 total
Time:        6.832 s
Ran all test suites.
❖ PS D:\Study\Politech\25-26\2_semestr\NodeJs\Lab5\dotaHeroes>

```

Рис.7.8 – Виконання тестів.

Перевіримо виконання у Postman.

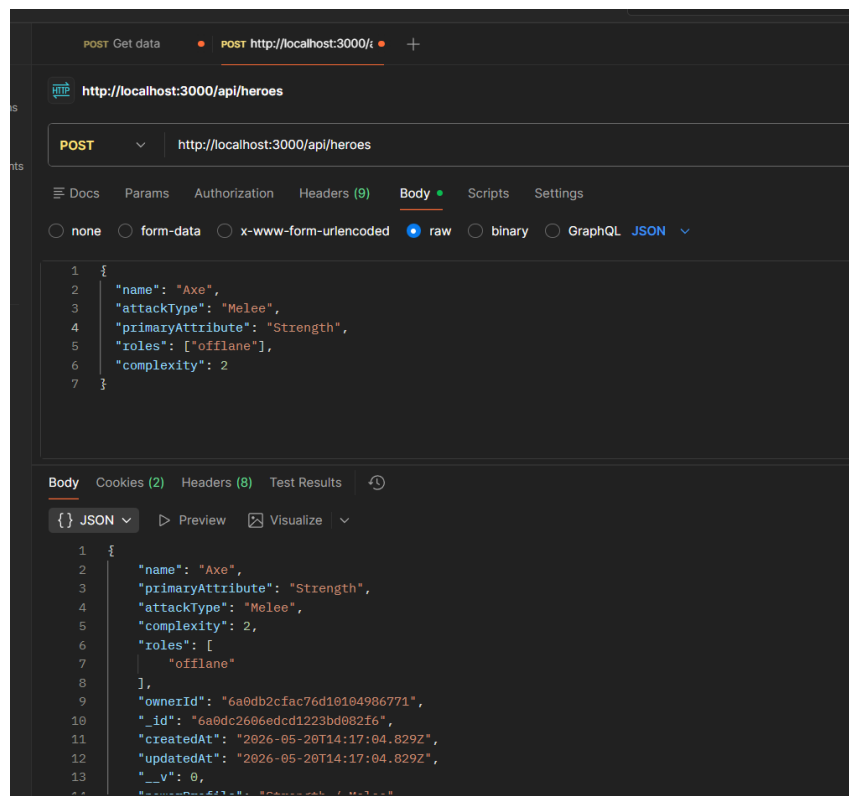


Рис.7.9 – Створення героя користувачем

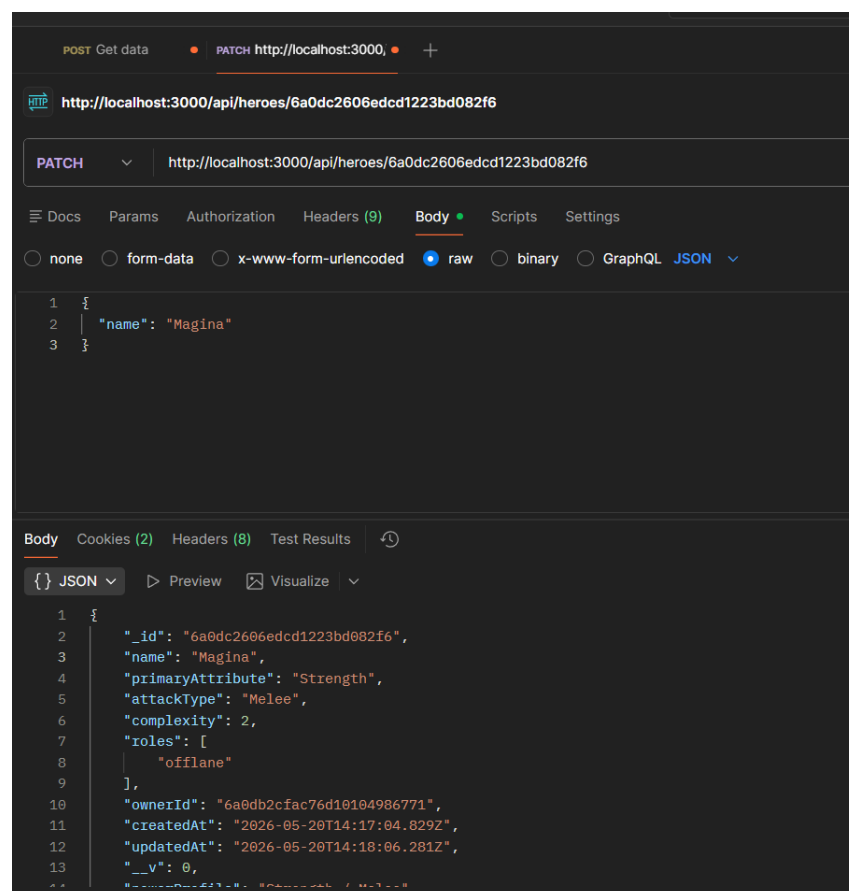


Рис.7.10 – Редагування героя користувачем

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.7	Арк.
						20
Змн.	Арк.	№ докум.	Підпис	Дата		

Увійдемо під іншим користувачем і перевіримо обмеження.

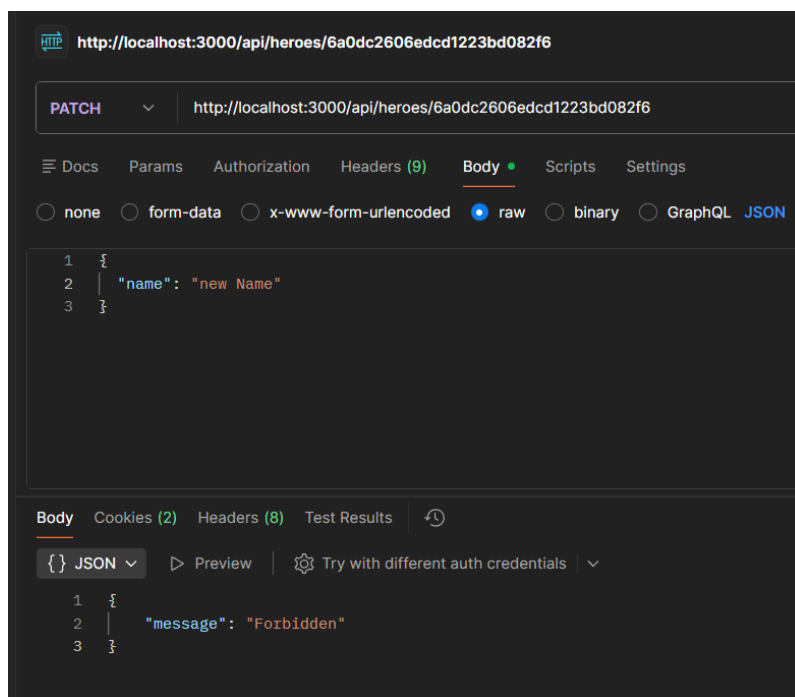


Рис.7.11 – Відмова в доступі для не власника

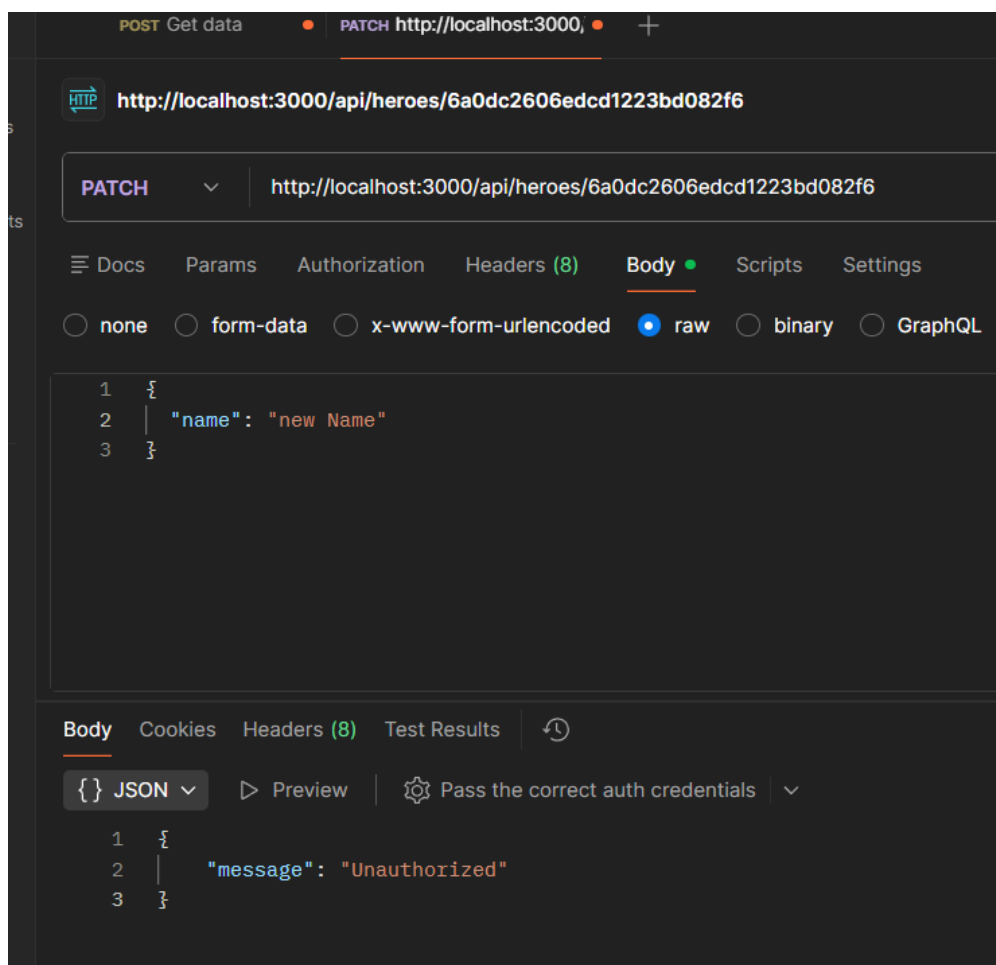


Рис.7.12 – Запит не авторизованим користувачем

Так як fly.io дає безкоштовний доступ на 7 днів, то проект немає змоги задеплойти, відповідно і CI/CD буде показувати помилку, адже намагається передеплойти проект після пушу.

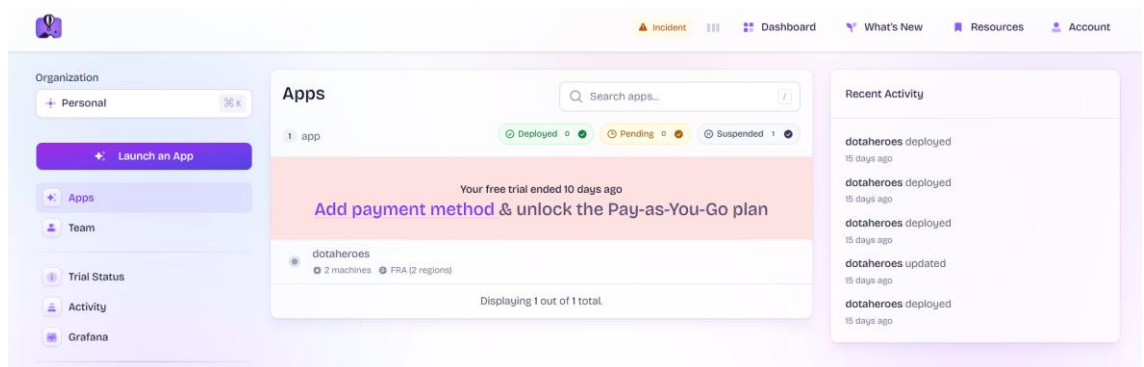


Рис.7.13 – Кінець trial версії fly.io

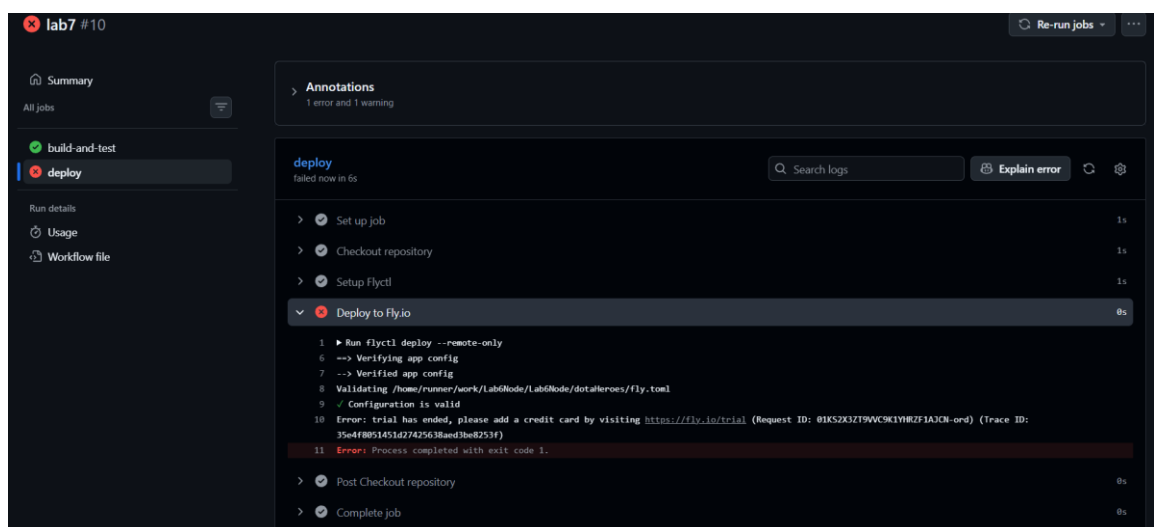


Рис.7.14 – Помилка деплою через кінець trial версії fly.io

Висновок: Під час виконання лабораторної роботи було набуто практичних навичок реалізації автентифікації та авторизації у Node.js-застосунках шляхом розширення застосунку з лабораторної роботи №6.